

22-Feb-2025

Object Oriented Programming

16- classes

- Classes are a template for creating objects.

class keyword se class banai jati hai

ex.

```
class Cricketer {  
  constructor (name, role) {  
    this.name = name  
    this.role = role  
    this.matchesPlayed = 0  
    this.stamina = 100  
  }  
}
```

we methods share
Ho rahi hai memory
ke ander

```
introduce () {  
  return `${this.name} the ${this.role} | matches Played:   
    ${this.matchesPlayed} | stamina: ${this.stamina}`  
}
```

Dono
ke
pass
alg
this
ka
context
hai
to share
ho
sahi
hai

Created a new players with new keyword

```
const player1 = new Cricketer ("Virat", "Batsman");  
const player2 = new Cricketer ("Bumrah", "Bowler");
```

```
console.log (player1. hasOwnProperty ("name"));  
console.log (typeof Cricketer);
```

uski kiski property
hai ya nahi

Output -
true
function
→ class ka data type function hota hai

Note :- classes are just the syntactic sugar, and the data type of is function

```
Console.log(player1.introduce());
```

```
Console.log(player2.introduce());
```

- player1 or player2 dono ke pass alag alag this ka context hai par vo share ho raha hai
- introduce method ha vo share ho rahi hai memory me, jab inko call kiya jayga tab inko inka reference or this pass kiya jayega

→ isliye isme difference hai

Output

Virat the Batsman | matchesPlayed: 0 | stamina: 100

Bumrah the Bowler | matchesPlayed: 0 | stamina: 100

Interview question

"gotchas"

- Data type of classes are function
- classes syntactic sugar hai

ex,

```
class Debutant {
```

```
  constructor(name) {
```

```
    this.name = name
```

```
    this.walkOut = () => `${this.name} walks  
    out to bat for the first time`
```

```
  }
```

```
}
```

Note:- Constructor is a special method used to create and initialise an object instance of class.

Method = A function that belongs to an object or a class.

```
const debutant1 = new Debutant("shubman")
const somethingfromLastClass = debutant1.walkOut();
```

```
console.log(somethingfromLastClass);
```

↓
detached method

output
shubman walks out for the first time.

- code runs because name in the same scope

```
const debutant2 = new Debutant("Yashasvi");
console.log(debutant1.walkOut === debutant2.walkOut);
```

output
false

- because debutant1 or debutant2 ke pass alg alg reference hai some memory ke isliye false.
this alg alg hai, shared memory hai.

- Side topic

17. Symbol - Data type.

ex (aadhar card Kyki uska number unique hota hai
har insaan ka)

- It is a primitive data type has unique and immutable value, often used to create non-conflicting property keys for object.

ex,

```
const aadhar_of_mayur = Symbol("aadhar");
```

```
const aadhar_of_piyush = Symbol("aadhar");
```

```
console.log(typeof aadhar_of_mayur);
```

output $\Rightarrow$ Symbol

```
console.log(aadhar_of_mayur === aadhar_of_piyush);
```

output $\Rightarrow$ false

- Symbol hamesh unique value deta hai, label same ho sakta hai par value hamesh unique hi hogi
- It's "guaranteed" unique value.

```
console.log(aadhar_of_mayur.toString());
```

output $\Rightarrow$ ~~aadhar~~ Symbol(aadhar)

- the `toString()` method for a symbol value is used to explicitly obtain a string representation of the symbol.

Note: symbols cannot be implicitly converted to strings through concatenation or template literals, it can throw a **TypeError**

```
console.log(aadhar_of_mayur.description);
```

Output $\Rightarrow$ aadhar

- the `.description` property of symbol instance is a read-only string that contain the description when symbol was created.
- It is used for debugging and identification purposes.
- Symbol ka labelled value property hai.

Note = toString() ki value symbol me nahi dikhati.
- vahi iski superpower hai

Date _____
Page _____

Interview question

ex,

```
const nonIndian = Symbol();  
console.log(nonIndian.description);
```

output

undefined

- Kyu Kyuki Symbol ke under value he hi nahi

isliye undefined.

ex,

```
const biometricHash = Symbol("biometricHash");
```

```
const bloodGroup = Symbol("bloodGroup");
```

```
const citizenRecord = {
```

```
  name: "Ved Pandey",
```

```
  age: 21,
```

```
  [biometricHash]: "a7yKnfkY788dn",
```

```
  [bloodGroup]: "O+",
```

```
};
```

Kar sakte hai

```
Object.keys(citizenRecord);
```

Object ke sare keys ko print karu.

```
console.log(Object.keys(citizenRecord));
```

Output

```
['name', 'age']
```

- Sirf do ki Key print hui Kyu Ki agar aap symbol banate ho or use as a key pass kar sakte ho to vo nahi dikhegi matlab print nahi hogi. visible property nahi hoti hai.

Note:- Object pe direct iteration nahi kar sakte

Date _____
Page _____

- agar aap ko symbol ke key ke ki value dekhni ho to, uske liye bhi ek method hai

(library or framework banate smay kam aata hai)

```
console.log(Object.getOwnPropertySymbols(citizenRecord))
```

output → [Symbol(biometricHash), Symbol(bloodGroup)]

- getOwnPropertySymbols()

This method returns an array containing all symbol properties defined directly on the object, and does not include string-based properties or inherited property

ex,

```
const governmentScheme = {  
  name: "PM Kisan Yojna",  
  people: 54,  
  [Symbol.toPrimitive](hint) {  
    if (hint === "string") return this.name;  
    if (hint === "number") return this.people;  
  }  
};
```

JavaScript apne ap hint value deti hai

Method that converts an object to a corresponding primitive value.

```
console.log(+governmentScheme);
```

pure governmentScheme ko number me 't' use karke convert

```
console.log(+governmentScheme);
```

Karte hai

pure governmentScheme ko string me String interpolation se Karte hai

output

88

PM Kisan Yojna

Go back to classes.

- classes nikahti hai
function ke upper syntactic sugar

18- Error Handling

(error ko gracefully handle karna, jo code run-
bhara se hai chalega to chalega nahi chalega
to nahi)

ex - agar aap ne database connection ke liye
request bheji, database connection nahi ho paya
to vaha system ko crash thodi na kar denge,
ek error message throw kar denge apni
tauf se, ki nahi ho paya ji database connect
error tha, usko hi error handling kehte

```
function bootNavigation (mapLoaded) {
  try {
    console.log('Is Navigation loaded: ' + mapLoaded)
    if (!mapLoaded) {
      throw new Error("Map was not passed in the function")
    }
    return "NAV-OK"
  } catch (error) {
```

eske ign kuthn bhi le sakte ho
cor error

```

console.log(error)
console.log('Navigation failed: ${error.message}')
} finally {
    console.log("Navigation sequence completed")
}
}

```

error print karna hai to false kar do

```

const status = bootNavigation(true);
console.log('Result: ${status}');

```

output

Is Navigation loaded: true

Navigation sequence completed

Result: NAV-OK

- ~~error file~~ errors ke type bhi defined kiye jate hai, or pata karate hai stack me se ki konse error aaga fir uske nisab se messages dete hai, error file usely centralized hoti hai

- then() - code chalaya true print kardo
- catch() - error aaye to print kardo
- finally() - fulfillment aakhri me chal hi jata hai

19 - Promises

- JavaScript single thread hai, ek hi thread pe ~~sara~~ kam hota hai, puri ki puri ~~sync~~ synchronous hai

Synchronous

- the default line by line code execution where each operation must complete before the next one begins

- To ham banate hai future me promise, ki network call karke ek promise mang lete hai, bhai dekh, tere pe vishwas kar raha hu, jab data aaye tab data dena, or apna kam karne lg jate hai, yahi promise hota hai

- promise se pehle callbacks the.

Callback

- callback is a function
`setTimeout()`
- it sets a timer which executes a function or specified piece of code once the timer expires
- Callback me sabse pehle error handle karte hai

- Reference ke liye callback nahi use krna chahiye.

```
function prepareOrderCB (dish, cb) {
  setTimeout(() => cb(null, { dish, status: "Prepared" }
    // error nahi denata
  ), 100)
}
```

itne time ke bad code chal jayega

```
function pickupOrderCB (order, cb) {
  setTimeout(() => cb(null, { ...order,
    // (dish) status: "picked-up" }, 100)
  )
}
```

function create

```
function deliverOrderCB (order, cb) {
  setTimeout(() => cb(null, { ...order,
    status: "delivered" }, 100)
  )
}
```

```
prepareOrderCB("Biryani", (err, order) => {
  if (err) return console.log(err)
  // Callback
  pickupOrderCB (order, (err, order) => {
    if (err) return console.log(err)
    // Callback
    deliverOrderCB (order, (err, order) => {
      if (err) return console.log(err);
      console.log(` ${order.dish} : ${order.status} `);
    });
  });
});
```

function consume

Three state of Promises

1) Pending

2) fulfilled

3) rejected

Promise has two part resolve and reject

ex,

```
function prepareOrder (dish) {
```

```
  return new Promise ((resolve, reject) => {
```

```
    setTimeout(() => {
```

```
      if (!dish) {
```

error throw

```
        reject (new Error ("No dish is there"))
```

kar do

```
      } else {
```

dish na

```
        return
```

hui to

```
      }
    })
  })
  console.log ('$$dish is ready')
  resolve ({ dish, status: "prepared" })
}
}
```

Creation
of promises

```
function pickupOrder (order) {
```

```
  return new Promise ((resolve, reject) => {
```

```
    setTimeout(() => {
```

```
      console.log ('$$order is ready')
```

```
      resolve ({ ...order, status: "picked up" })
```

```
    }, 100)
```

```
  })
```

```
}
```

function deliverOrder(order) {
 return new Promise((resolve, reject) => {
 setTimeout(() => {
 console.log(`\${order} is ready`)
 resolve({...order, status: "delivered"})
 }, 100)
 })
}

Creation
of
Promise

prepareOrder("Paneer Tikka")

• then(order => pickupOrder(order))

• then(order => deliverOrder(order))

} • then
success case
handle kardo
hai

• catch() ← error aaye to catch kardo

→ Consumption of Promises

Javascript or DOM (Document Object Model)
alag alag chiz hai

Javascript core language hai

DOM one the implementation hai

Fetch + f bhi ek implementation hai